A Markov Chain Monte Carlo Method for Efficient Finite-Length LDPC Code Design

Ata Tanrıkulu, Mete Yıldırım, and Ahmed Hareedy, *Member, IEEE*

***Abstract*—Low-density parity-check (LDPC) codes are among the most prominent error-correction schemes in today's data-driven world. They find application to fortify various modern storage, communication, and computing systems. Protograph-based (PB) LDPC codes offer many degrees of freedom in the code design and enable fast encoding and decoding. In particular, spatially-coupled (SC) and multi-dimensional (MD) circulant-based codes are PB-LDPC codes with excellent performance. Efficient finite-length (FL) algorithms are required in order to effectively exploit the available degrees of freedom offered by SC partitioning, lifting, and MD relocations. In this paper, we propose a novel Markov chain Monte Carlo (MCMC or MC^2) method to perform this FL optimization, addressing the removal of short cycles. While we focus on partitioning and lifting of SC codes, our MC^2 approach can effectively work for other procedures and/or other code designs. While iterating, we draw samples from a defined distribution where the probability decreases as the number of short cycles from the previous iteration increases. We analyze our MC^2 method theoretically as we prove the invariance of the Markov chain where each state represents a possible partitioning or lifting arrangement, i.e., sample, that has a specific probability. Via our simulations, we then fit the distribution of the number of cycles resulting from a given arrangement on a Gaussian distribution. By analyzing the mean, we derive estimates for cycle counts that are close to the actual counts. Furthermore, we derive the order of the expected number of iterations required by our MC^2 approach to reach a local minimum as well as the size of the Markov chain recurrent class through approximating the probability of getting arbitrarily close to the local minimum. Our approach is compatible with code design techniques based on gradient-descent. Experimental results show that our MC^2 method generates SC codes with remarkably fewer short cycles and substantial gains in error/erasure-rate performance compared with the current state-of-the-art. Moreover, to reach the same number of cycles, our MC^2 method requires orders of magnitude less overall time compared with the available literature methods.**

***Index Terms*—LDPC codes, MCMC methods, spatially-coupled codes, finite-length optimization.**

I. Introduction

SINCE their introduction by Gallager in 1963 [1] then their reinvention by MacKay in 1999 [2], low-density parity-check (LDPC) codes have been increasingly gaining ground as the error-correction scheme of choice in various modern systems. Today, LDPC modules exist in various data storage, data transmission, as well as digital communication systems [3], [4], [5], [6], [7] because of their capacity-approaching performance and their practical message-passing decoding [2]. Protograph-based LDPC codes are designed from a base matrix (graph), called a protograph matrix (protograph), that is lifted via circulant permutation matrices (CPMs) or, in short, circulants [8], [9]. In addition to the degrees of freedom they offer the code designer via the circulant powers, these codes also support fast and efficient encoding and decoding procedures. In 2004, Fossorier introduced algebraic conditions on the circulant powers, which specify the circular shift of each CPM diagonal elements, to maintain or remove a protograph cycle after lifting [8]. Throughout this paper, we interchangeably use the terms "matrix" and its corresponding "graph" when the context is clear.

Recent innovations in LDPC code design and decoding led to the introduction of spatially-coupled (SC) [9], [10] and multi-dimensional (MD) codes [11], [12]. SC codes are designed by partitioning an underlying block matrix and coupling the components [13], [14]. MD codes are designed by relocating entries from one-dimensional LDPC code copies to connect them [12]. SC codes offer capacity-achievability [15], [16], low-latency if windowed decoding is adopted [17], [18], as well as additional degrees of freedom for finite-length code design coming from partitioning. MD codes offer even further flexibility and they can mitigate (multi-dimensional) channel non-uniformity [11]. To better exploit the degrees of freedom for SC and MD codes, we recently introduced a probabilistic approach that is based on the gradient-descent (GD) algorithm to efficiently design locally-optimal codes with respect to minimizing the multiplicities of detrimental cycles/objects in their graph representations [19], [20]. Detrimental cycles/objects here are small ones that are either dominant absorbing sets [21] or common subgraphs of dominant absorbing sets [22].

Protograph-based SC (MD) code design procedure consists of two (three) stages, partitioning and lifting (partitioning, lifting, and relocations) [10], [12], [22], [23], [24], [25]. With or without probabilistic approaches in the design, finite-length (FL) algorithmic optimization is necessary for all the aforementioned stages. For example, various circulant-power optimizers (CPOs) were presented to perform the stage of lifting [10], [22]. There are two metrics to gauge the efficiency of an FL algorithmic optimizer (AO). The first is performance,

Received 12 August 2025; revised 17 October 2025; accepted 13 December 2025. Date of publication 23 December 2025; date of current version 17 March 2026. This work was supported in part by the TÜBİTAK 2232-B International Fellowship for Early Stage Researchers. The associate editor coordinating the review of this article and approving it for publication was D. Vukobratovic. *(Corresponding author: Ahmed Hareedy.)*

The authors are with the Department of Electrical and Electronics Engineering, Middle East Technical University (METU), 06800 Ankara, Türkiye (e-mail: ata.tanrikulu@metu.edu.tr; mete.yildirim@metu.edu.tr; ahareedy@metu.edu.tr).

Digital Object Identifier 10.1109/TCOMM.2025.3647737

measured by how much it can reduce the number of target cycles/objects. The second is execution time, measured by how many iterations required to reach some count. As the degrees of freedom in the SC or MD code design increase, the complexity of existing FL-AOs grows rapidly.

Markov chain Monte Carlo (MCMC) methods are effective methods to draw samples from a specific probability distribution based on some metric [26], [27], [28], and they can be useful in discrete optimization [29], [30]. In this paper, we offer the first MCMC framework, which we refer to as our MC^2 method, that effectively performs FL optimization to design high performance LDPC codes. In particular, the proposed MC^2 method can reach locally-optimal parameters for SC partitioning, MD relocations, and circulant-power lifting in remarkably less computational time compared with the available state-of-the-art. Optimality here is with respect to minimizing the number of short cycles or small graphical objects (here, these are also cycles with specific properties, and they are common subgraphs of dominant detrimental objects [12], [22]). Without loss of method generality, we focus on the design of SC and MD-SC codes in this work.

In our MC^2 method, the samples drawn represent partitioning or lifting arrangements, and the distribution is constructed such that the probability is inversely proportional to the total number of cycles of interest. The core contributions of the proposed MC^2 method are summarized as follows:

- We provide a theoretical analysis proving the invariance and aperiodicity properties of the Markov chain defined by our MC^2 sampling process.
- Through simulations, we collect data regarding the number of cycles and fit the resulting histogram to a Gaussian distribution.
- We derive a differential equation where the rate of change of the Gaussian mean of the cycle count is expressed as a function of its variance.
- Using the fitted model and differential equation, we estimate the number of cycles, obtaining values close to the actual observed cycle counts after optimization.
- We approximate the probability that the cycle count lies within an arbitrary ϵ of the local optimum, and use this probability to estimate:
  - the order of the number of iterations required to reach the local optimum, and
  - the number of states in the recurrent class of the Markov chain.
- We offer experimental results that demonstrate:
  - significant reductions in the number of cycles of lengths 6 and 8 compared with existing FL-AOs,
  - orders-of-magnitude improvements in computational time to reach comparable or better cycle counts, and
  - frame error/erasure-rate performance gains of up to 1.61 orders of magnitude.
- We show that when the local optimum search space is reduced using the GD algorithm [19], our MC^2 method achieves even greater performance improvements.
- With this method, we offer code designers an FL optimization algorithm that can outperform other heuristic FL algorithms [10], [19], while also enabling the design of SC codes with high memory, which is beyond the practical limits of optimal methods [10].

The rest of the paper is organized as follows. Preliminaries about SC code construction and MCMC methods are presented in Section II. Our MC^2 probabilistic optimizer (the framework and the algorithm) is introduced in Section III. Data-fitting-based estimation of the cycle count, iteration number, and recurrent class size is discussed in Section IV. Numerical results on the cycle counts, computational times, estimation results, and error/erasure-rate performance for various codes are presented in Section V. Finally, the paper is concluded in Section VI.

II. Preliminaries

A. Spatially-Coupled Codes

Let $\mathbf{H}_{\text{SC}}$ in (1) be the parity-check matrix of a circulant-based (CB) spatially-coupled (SC) code with parameters $(\gamma, \kappa, z, L, m)$, where $\gamma \geq 3$ and $\kappa > \gamma$ are the column and row weights of the underlying block code, respectively. The SC coupling length and memory are L and m, respectively.

$$\mathbf{H}_{\text{SC}} = \begin{bmatrix} \mathbf{H}_0 & \mathbf{0} & & & \mathbf{0} \\ \mathbf{H}_1 & \mathbf{H}_0 & & & \vdots \\ \vdots & \mathbf{H}_1 & \ddots & & \vdots \\ \mathbf{H}_m & \vdots & \ddots & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{H}_m & \ddots & \ddots & \mathbf{H}_0 \\ \vdots & \mathbf{0} & & \ddots & \mathbf{H}_1 \\ \vdots & \vdots & & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & & & \mathbf{H}_m \end{bmatrix}. \tag{1}$$

$\mathbf{H}_{\text{SC}}$is obtained from a binary matrix $\mathbf{H}^{\text{g}}_{\text{SC}}$ by replacing each nonzero (zero) entry in the latter by a circulant (zero) $z \times z$ matrix, $z \in \mathbb{N}$. The notation "g" in the superscript of any matrix refers to its protograph matrix, where each circulant matrix is replaced by 1 and each zero matrix is replaced by 0. $\mathbf{H}^{\text{g}}_{\text{SC}}$ is called the protograph matrix of the SC code, and it has the same convolutional structure in (1) composed of L replicas $\mathbf{R}^{\text{g}}_i$ of size $(m+L)\,\gamma \times \kappa$ each, where

$$\mathbf{R}^{\text{g}}_i = \left[(\mathbf{0}_{\gamma i \times \kappa})^{\text{T}} (\mathbf{H}^{\text{g}}_0)^{\text{T}} \; (\mathbf{H}^{\text{g}}_1)^{\text{T}} \; \ldots \; (\mathbf{H}^{\text{g}}_m)^{\text{T}} \; (\mathbf{0}_{\gamma(L-1-i)\times\kappa})^{\text{T}}\right]^{\text{T}},$$

and $\mathbf{H}^{\text{g}}_y$'s are all of size $\gamma \times \kappa$, and are pairwise disjoint. The sum $\mathbf{H}^{\text{g}} = \sum_{y=0}^{m} \mathbf{H}^{\text{g}}_y$ is called the base matrix, which is the protograph matrix of the underlying block matrix $\mathbf{H} = \sum_{y=0}^{m} \mathbf{H}_y$. In this paper, $\mathbf{H}^{\text{g}}$ is taken to be all-one matrix, and the SC codes have quasi-cyclic (QC) structure.

Here, the $\gamma \times \kappa$ matrix $\mathbf{P}$ whose entry at (i, j) is $\mathbf{P}(i, j) = a \in \{0, 1, \ldots, m\}$ when $\mathbf{H}^{\text{g}}_a = 1$ at that entry is called the partitioning matrix. The $\gamma \times \kappa$ matrix $\mathbf{L}$ with $\mathbf{L}(i, j) = f_{i,j} \in \{0, 1, \ldots, z-1\}$, where $\sigma^{f_{i,j}}$ is the circulant in $\mathbf{H}$ lifted from the entry $\mathbf{H}^{\text{g}}(i, j)$, is called the lifting matrix. Here, σ is the $z \times z$ identity matrix with its columns cyclically shifted one unit to the left. Observe that $m+1$ is the number of component matrices $\mathbf{H}^{\text{g}}_y$ (or $\mathbf{H}_y$), and L is the number of replicas in $\mathbf{H}^{\text{g}}_{\text{SC}}$ (or $\mathbf{H}_{\text{SC}}$).

A special class of SC codes is the class of topologically-coupled (TC) codes [19], characterized by pseudo-memory features. In TC codes, a deliberate design choice is to set a subset of component matrices to be entirely zero. The terminology "topologically-couple" stems from the topological degrees of freedom available to the code designer in selecting the non-zero component matrices.

A cycle-$2g$ candidate in $\mathbf{H}_{\text{SC}}^{\text{g}}$ is a way of traversing a structure to generate cycles of length $2g$ after lifting [22]. Similarly, we define object candidates as ways of traversing structures in the protograph to generate objects after lifting. The same concepts also apply to $\mathbf{H}^{\text{g}}$ for partitioning [13]. In an SC code, each cycle in the Tanner graph of $\mathbf{H}_{\text{SC}}$ corresponds to a cycle candidate in the protograph matrix $\mathbf{H}_{\text{SC}}^{\text{g}}$, and each cycle candidate in $\mathbf{H}_{\text{SC}}^{\text{g}}$ corresponds to a cycle candidate $\mathcal{C}$ in the base matrix $\mathbf{H}^{\text{g}}$.

Lemma 1 specifies a necessary and sufficient condition for a cycle candidate in $\mathbf{H}^{\text{g}}$ to become a cycle candidate in the SC protograph and then a cycle in the final SC Tanner graph.

Lemma 1: Consider an SC code constructed as illustrated above. Let $\mathcal{C}$ be a cycle-$2g$ candidate in the base matrix, where $g \in \mathbb{N}$ and $g \geq 2$. Denote $\mathcal{C}$ by $(i_1, j_1, i_2, j_2, \ldots, i_g, j_g)$, where (i_k, j_k) and (i_k, j_{k+1}), $1 \leq k \leq g$, $j_{g+1} = j_1$, are nodes (entries) of $\mathcal{C}$ in $\mathbf{H}^{\text{g}}$. The partitioning and lifting matrices are $\mathbf{P}$ and $\mathbf{L}$, respectively. Then $\mathcal{C}$ becomes a cycle-$2g$ candidate in the SC protograph, i.e., remains active, if and only if the following condition follows [13]:

$$\sum_{k=1}^{g} \mathbf{P}(i_k, j_k) = \sum_{k=1}^{g} \mathbf{P}(i_k, j_{k+1}). \tag{2}$$

This cycle candidate becomes a cycle-$2g$ in the SC Tanner graph, i.e., remains active, if and only if [8]:

$$\sum_{k=1}^{g} \mathbf{L}(i_k, j_k) \equiv \sum_{k=1}^{g} \mathbf{L}(i_k, j_{k+1}) \pmod{z}. \tag{3}$$

B. Markov Chain Monte Carlo Methods

Markov chains are discrete-time stochastic processes that probabilistically undergo transitions from one state to another within a state space. They are characterized by the property that given the current state, the future state becomes independent of the past states. Monte Carlo methods are computational algorithms that rely on repeated random sampling to obtain numerical results. They are particularly useful in calculating integrals, simulating systems with inherent randomness, and emulating high-dimensional systems.

Markov chain Monte Carlo (MCMC) methods integrate the concepts of Markov chains and Monte Carlo techniques. These methods are used to sample from sophisticated probability distributions, which is particularly useful in high-dimensional spaces where direct sampling is challenging. MCMC methods require two main conditions for success: the invariance of the distribution and the ergodicity of the Markov chain or its recurrent class of interest [26]. Invariance ensures that if the chain starts from the stationary distribution, it remains in this distribution at every step (iteration). Ergodicity guarantees that the chain can reach any part of the state space in a finite number of steps, which implies that the sampling distribution converges to the stationary distribution as more samples are drawn, i.e., more iterations are performed.

One specific MCMC technique of interest is the Metropolis-Hastings algorithm, which allows sampling from a distribution by generating a sequence of sample values that effectively explore the target distribution [27], [28]. This method involves suggesting moves in the state space, which are the samples, according to a proposed distribution and accepting or rejecting each move based on the acceptance probability that ensures invariance to appropriately represent the target distribution.

Another technique is Gibbs sampling, a special case of the Metropolis-Hastings algorithm, particularly used when the conditional probabilities of each variable, given all other variables, are easy to compute [32]. These variables constitute the state. In Gibbs sampling, we access each variable to sample from its conditional distribution:

$$x_i^{(t+1)} \sim P\left(x_i \mid x_1^{(t+1)}, \ldots, x_{i-1}^{(t+1)}, x_{i+1}^{(t)}, \ldots, x_n^{(t)}\right), \tag{4}$$

where x_i is a component of the state vector $\mathbf{x}$ of length n and t denotes the time-step. This method is more feasible when conditional distributions are straightforward to calculate, making it particularly effective for advanced, high-dimensional distributions. Traditionally used to update one variable at a time based on its conditional probabilities relative to other variables, the Gibbs sampler can be adapted to update multiple variables simultaneously if their joint conditional probabilities are computationally tractable.

III. Probabilistic Optimizer Design

In this section, we introduce our optimization problem, which is the general case of short-cycle or common-substructure minimization in the SC code design at the partitioning and lifting stages.[1] We also present our proposed Gibbs sampling-based MC^2 finite-length (FL) optimization method.

A. Optimization Problem Overview

We define two separate optimization problems for the partitioning and the lifting stages of SC code design. Our goal is to find optimal or suboptimal partitioning and lifting matrices to reduce the number of detrimental objects of interest. We denote the input or state vector by $\mathbf{x}$, which is obtained by concatenating the rows of either the partitioning or the lifting matrix, depending on the problem under consideration. We define $C(\mathbf{x})$ as the objective function, which is the total number of detrimental objects under this $\mathbf{x}$, and F as the feasible set of the problem.

The reason these two stages of code design are separated is that joint design does not consistently result in better solutions compared with separate design, and the former significantly increases the complexity of the design process [10], [23].

At the partitioning stage, we focus on minimizing the count of short cycle candidates in $\mathbf{H}_{\text{SC}}^{\text{g}}$ by focusing on short cycle

[1] A common substructure is a common subgraph of various absorbing sets that dominate the error profile of the code when the channel conditions are better [12], [22].

candidates in the base matrix. The optimization problem is formulated as follows:

$$\underset{\mathbf{x}}{\text{minimize}}\; C(\mathbf{x}) = w_4\, C_4(\mathbf{x}) + w_6\, C_6(\mathbf{x}) + w_8\, C_8(\mathbf{x}), \quad (5)$$

where $\{0, 1, \ldots, m\}^{\gamma\kappa}$ is the domain and the feasible set. $C_4(\mathbf{x})$, $C_6(\mathbf{x})$, and $C_8(\mathbf{x})$ represent the number of cycle-4, cycle-6, and cycle-8 candidates in the base matrix $\mathbf{H}^{\text{g}}$ that remain active given the partitioning specified by $\mathbf{x}$, and w_4, w_6, and w_8 are their corresponding weights. We incorporate the probabilistic framework of [19], which is based on gradient-descent (GD), to approach this problem as we use the output from the GD distributor to guide partitioning. The MC^2 FL optimizer is initialized using this output.

For the lifting stage, the objective is to minimize the occurrence of short cycles or common substructures of interest while keeping smaller cycles and/or low-weight codewords inactive in the final Tanner graph of the SC code. The problem is defined as:

$$\begin{aligned} &\underset{\mathbf{x}}{\text{minimize}}\; C(\mathbf{x}) \\ &\text{subject to no active objects in } \mathcal{L}', \end{aligned} \quad (6)$$

where $\{0, 1, \ldots, z-1\}^{\gamma\kappa}$ is the domain and $C(\mathbf{x})$ is the number of objects in $\mathbf{H}^{\text{g}}_{\text{SC}}$ that remain active after lifting is done based on $\mathbf{x}$. $\mathcal{L}'$ is the list of smaller objects that we wish to keep inactive, and the feasible set F represents the subset of the domain where each element ensures that all objects in the set $\mathcal{L}'$ remain inactive.

B. Theoretical Framework

In our derivations, we focus on a normalized version of the objective function defined as:

$$\overline{C}(\mathbf{x}) = \frac{C(\mathbf{x})}{\alpha}, \quad (7)$$

where α is the normalizing factor, and it is equal to the maximum value that the objective function could take. From this point forward, we will refer to the normalized version as the objective function for brevity.

Definition 1 (Main Distribution): The joint probability distribution

$$P(\mathbf{x}) = \frac{P^*(\mathbf{x})}{Z(\beta)} \quad (8)$$

of the input vector $\mathbf{x}$*, where*

$$P^*(\mathbf{x}) = \begin{cases} e^{-\beta\overline{C}(\mathbf{x})}, & \text{if } \mathbf{x} \in F, \\ 0, & \text{otherwise,} \end{cases} \quad (9)$$

is defined as the main distribution. Here, $Z(\beta)$ *is the normalizing factor*

$$Z(\beta) = \sum_{\mathbf{x}\in F} e^{-\beta\overline{C}(\mathbf{x})} \quad (10)$$

and $\beta \in \mathbb{R}^{\geq 0}$ *is a hyper-parameter.*

The probability value of the main distribution is inversely proportional to the objective function, which results in highest probabilities being assigned to near-optimal and optimal solutions of our problem. While computing $P^*(\mathbf{x})$ for any $\mathbf{x} \in F$ requires only a single evaluation, computing $Z(\beta)$ necessitates $|F|$ evaluations of the objective function. This scales as $O\left(N^{\gamma\kappa}\right)$, where N represents the number of possible values for each entry of $\mathbf{x}$. Thus, evaluating $P(\mathbf{x})$ is computationally challenging. Our primary goals are to generate samples from this computing-wise challenging distribution using Monte Carlo methods and to exploit the proportionality between optimal results and their corresponding high probabilities so that we can build an efficient MC^2 FL optimizer.

The conditional probability of a subset of entries of $\mathbf{x}$ given the remaining entries, under the main distribution, is:

$$P\left(\mathbf{x}_\nu \mid \mathbf{x}_{\setminus\nu}\right) = \frac{P(\mathbf{x})}{\sum_{\mathbf{x}':\ \mathbf{x}'_{\setminus\nu}=\mathbf{x}_{\setminus\nu}} P(\mathbf{x}')} = \frac{P^*(\mathbf{x})}{\sum_{\mathbf{x}':\ \mathbf{x}'_{\setminus\nu}=\mathbf{x}_{\setminus\nu}} P^*(\mathbf{x}')}. \quad (11)$$

Here, ν denotes the set of selected indices, $\setminus\nu$ refers to the remaining indices, and $\mathbf{x}_\nu$ represents the subset of entries of $\mathbf{x}$ indexed by ν. Evaluating the probabilities for this conditional distribution requires $O\left(N^d\right)$ evaluations of the objective function, where $d = |\nu|$. For small d, this is computationally feasible, enabling the use of Gibbs sampling.

In our optimization algorithm, we construct a Markov chain where each state corresponds to an $\mathbf{x} \in F$, and transitions are based on conditional probabilities from Equation (11). A list of d-tuples of indices is maintained whose length is same as that of the input vector $\mathbf{x}$. Each tuple on this list involves one of the indices, say ζ, from the set $\{0, 1, \ldots, |\mathbf{x}| - 1\}$ corresponding to the entries of $\mathbf{x}$ and the $d-1$ indices of the most correlated entries to the one indexed by ζ. Correlation is determined by the number of objects of interest that share these entries or, for a weighted objective function, the weighted sum of these numbers for different object types.

The algorithm iterates over the list of d-tuples, evaluating the conditional probability mass function (PMF) as the transition probability for the current index d-tuple using (11). The transition probability from the current state $\mathbf{x}$ to the next state $\mathbf{x}'$, given that the current (selected) index d-tuple is ν, is denoted by $T\left(\mathbf{x}'; \mathbf{x} \mid \nu\right)$ and has the below relation:

$$T\left(\mathbf{x}'; \mathbf{x} \mid \nu\right) = \begin{cases} P\left(\mathbf{x}'_\nu \mid \mathbf{x}_{\setminus\nu}\right), & \text{if } \mathbf{x}_{\setminus\nu} = \mathbf{x}'_{\setminus\nu}, \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

New states are reached, i.e., new samples are drawn, based on these transition probabilities, and the minimum objective function value as well as the corresponding input vector are updated whenever a better solution is found during iterative evaluations. To avoid repeated patterns, the list of d-tuples is randomly shuffled after each pass of the list.

Lemma 2: $P(\mathbf{x})$ *is an invariant distribution of the Markov chain constructed by the described sampling method above, and this Markov chain is aperiodic.*

Proof: Let $\mathbf{x}$ and $\mathbf{x}'$ be two states of the Markov chain. Here, t is defined as the tuple of indices at which the entries of these two vectors are different, and S is the list of d-tuples indexing entries that are updated during Gibbs sampling. We introduce $T\left(\mathbf{x}'; \mathbf{x}\right)$ as the overall transition probability from $\mathbf{x}$ to $\mathbf{x}'$ and $S^{\{t\}}$ as a sub-list of S defined as $S^{\{t\}} = \{t' \mid t \subseteq t', t' \in S\}$,

where $|t'| = d$. We can write transition probabilities as

$$T(\mathbf{x}';\mathbf{x}) = \sum_{\nu \in S} P(\nu) T(\mathbf{x}';\mathbf{x} \mid \nu) = \sum_{\nu \in S^{\{t\}}} P(\nu) P\left(\mathbf{x}'_{\nu} \mid \mathbf{x}_{\setminus\nu}\right), \tag{13}$$

where $P(\nu)$ is the probability that the d-tuple ν is selected as the tuple of indices of entries to be updated. Also, since $\mathbf{x}$ and $\mathbf{x}'$ only differ at indices that are elements of tuple t, $\forall \nu \in S^{\{t\}}$, $\mathbf{x}_{\setminus\nu} = \mathbf{x}'_{\setminus\nu}$, leading to $T(\mathbf{x}';\mathbf{x} \mid \nu) = P\left(\mathbf{x}'_{\nu} \mid \mathbf{x}_{\setminus\nu}\right) \neq 0$. We can then show that

$$\begin{aligned} T(\mathbf{x}';\mathbf{x})\,P(\mathbf{x}) &= \sum_{\nu \in S^{\{t\}}} P(\nu) P\left(\mathbf{x}'_{\nu} \mid \mathbf{x}_{\setminus\nu}\right) P(\mathbf{x}) \\ &= \sum_{\nu \in S^{\{t\}}} P(\nu) P\left(\mathbf{x}'_{\nu} \mid \mathbf{x}_{\setminus\nu}\right) P(\mathbf{x}_{\setminus\nu}) P\left(\mathbf{x}_{\nu} \mid \mathbf{x}_{\setminus\nu}\right) \\ &= \sum_{\nu \in S^{\{t\}}} P(\nu) P\left(\mathbf{x}'_{\nu} \mid \mathbf{x}'_{\setminus\nu}\right) P(\mathbf{x}'_{\setminus\nu}) P\left(\mathbf{x}_{\nu} \mid \mathbf{x}'_{\setminus\nu}\right) \\ &= \sum_{\nu \in S^{\{t\}}} P(\nu) P\left(\mathbf{x}_{\nu} \mid \mathbf{x}'_{\setminus\nu}\right) P(\mathbf{x}') \\ &= T(\mathbf{x};\mathbf{x}')\,P(\mathbf{x}'). \end{aligned} \tag{14}$$

Resulting relation (14) is known as detailed balance, which implies the invariance of the distribution of $P(\mathbf{x})$ under this Markov chain.

Lastly, from Equations (11) and (13), it can be seen that $T(\mathbf{x};\mathbf{x}) > 0$, $\forall \mathbf{x} \in F$. The ability to self-transition makes the Markov chain aperiodic. ■

As outlined in Section II, for an MCMC method to converge to the desired distribution, the associated Markov chain must be ergodic (has a single recurrent class that is aperiodic) and the given distribution must be invariant from the chain. While Lemma 2 establishes aperiodicity and invariance, the presence of a single recurrent class remains unaddressed. Having said that, we only consider the recurrent class observed during our MC^2 algorithm as a small chain for which, all convergence conditions are satisfied, leaving further discussion regarding multiple recurrent classes for Section V.

Definition 2 (Main Distribution of a Recurrent Class): We denote the recurrent class that the MC^2 algorithm enters by A. The main distribution of A is

$$P_A(\mathbf{x}) = \begin{cases} \dfrac{e^{-\beta\,\overline{C}(\mathbf{x})}}{Z_A(\beta)}, & \text{if } \mathbf{x} \in A, \\ 0, & \text{otherwise}, \end{cases} \tag{15}$$

where

$$Z_A(\beta) = \sum_{\mathbf{x} \in A} e^{-\beta\,\overline{C}(\mathbf{x})} \tag{16}$$

is the normalizing factor for A.

Invariance of $P_A(\mathbf{x})$ can be easily shown by altering the proof of Lemma 2 because the definitions of conditional and transition probabilities remain the same, except for operating within A instead of F. Since the Markov chain simulation cannot leave this recurrent class once it enters and since this recurrent class is also aperiodic, the steady-state distribution of the main chain on this recurrent class (the small chain), $\pi_A(\mathbf{x})$, converges to $P_A(\mathbf{x})$.

Theorem 1: Let A be the recurrent class that the Gibbs sampler enters during the run-time of the algorithm. Then, as the sample count $t \to \infty$, the difference between the local minimum value that the objective function $\overline{C}(\mathbf{x})$ could take in this recurrent class A (denoted by $\overline{C}^*_A$) and the expected value of the objective function under the steady-state distribution of the Markov chain in this recurrent class is upper bounded by $\frac{1}{\beta}\ln\left(\frac{|A|}{|A^*|}\right)$, where $A^* = \left\{\mathbf{x} \mid \overline{C}(\mathbf{x}) = \overline{C}^*_A, \mathbf{x} \in A\right\}$.

Proof: For any $\mathbf{x}^* \in A^*$ and for the recurrent class of interest

$$\begin{aligned} \mathbb{E}_A\left\{e^{-\beta\left(\overline{C}(\mathbf{x}^*)-\overline{C}(\mathbf{x})\right)}\right\} &= \sum_{\mathbf{x} \in A} \pi_A(\mathbf{x})\, e^{-\beta\left(\overline{C}(\mathbf{x}^*)-\overline{C}(\mathbf{x})\right)} \\ &= \sum_{\mathbf{x} \in A} \left(\pi_A(\mathbf{x})\,\frac{\pi_A(\mathbf{x}^*)}{\pi_A(\mathbf{x})}\right) = \pi_A(\mathbf{x}^*)\,|A|. \end{aligned} \tag{17}$$

Since $\pi_A(\mathbf{x}^*)\,|A^*| \leq 1$, it follows that $\pi_A(\mathbf{x}^*)\,|A| \leq \frac{|A|}{|A^*|}$. Hence,

$$\mathbb{E}_A\left\{e^{-\beta\left(\overline{C}(\mathbf{x}^*)-\overline{C}(\mathbf{x})\right)}\right\} \leq \frac{|A|}{|A^*|}. \tag{18}$$

Using Jensen's inequality for convex functions, we get:

$$\begin{aligned} \frac{|A|}{|A^*|} &\geq \mathbb{E}_A\left\{e^{-\beta\left(\overline{C}(\mathbf{x}^*)-\overline{C}(\mathbf{x})\right)}\right\} = e^{-\beta\,\overline{C}^*_A}\,\mathbb{E}_A\left\{e^{\beta\,\overline{C}(\mathbf{x})}\right\} \\ &\geq e^{-\beta\,\overline{C}^*_A}\, e^{\beta\,\mathbb{E}_A\left\{\overline{C}(\mathbf{x})\right\}} = e^{\beta\left(\mathbb{E}_A\left\{\overline{C}(\mathbf{x})\right\}-\overline{C}^*_A\right)}. \end{aligned} \tag{19}$$

Thus, by taking natural logarithm,

$$\ln\left(\frac{|A|}{|A^*|}\right) \geq \beta\left(\mathbb{E}_A\left\{\overline{C}(\mathbf{x})\right\} - \overline{C}^*_A\right). \tag{20}$$

Consequently,

$$\mathbb{E}_A\left\{\overline{C}(\mathbf{x})\right\} - \overline{C}^*_A \leq \frac{1}{\beta}\ln\left(\frac{|A|}{|A^*|}\right), \tag{21}$$

which completes the proof. ■

This theorem indicates the convergence of the MC^2 algorithm to a near-optimal solution. As the sample count increases, the mean value of the objective function over the sampled states should converge to its expected value since the recurrent class entered is ergodic and satisfies invariance [26]. The minimum value obtained by the Markov chain should be less than this mean value. Hence, it should also be less than the expected value, as it falls within the range $\left[\overline{C}^*_A, \overline{C}^*_A + \frac{1}{\beta}\ln\left(\frac{|A|}{|A^*|}\right)\right]$. Observe that

$$\frac{1}{\beta}\ln\left(\frac{|A|}{|A^*|}\right) \leq \frac{1}{\beta}\ln(z^{\gamma\kappa}) = \frac{\gamma\kappa}{\beta}\ln z. \tag{22}$$

for the lifting problem.

Remark 1: Although increasing β appears to narrow this range and bring its maximum closer to the local minimum, it also makes transitions in the Markov chain more challenging and decreases the rate of convergence. This occurs because, with a higher β, states with lower objective function values gain significantly higher weights in the transition probabilities, making the algorithm more prone to greediness and potentially leading it to suboptimal solutions. This could increase the number of iterations required to achieve a satisfactory result.

At this stage, we define $f(\beta)$, the *acceptance rate*, as the ratio of the number of transitions between distinct states to the total number of transitions. This metric measures the flow of state transitions, with higher β making the Markov chain more greedy, increasing the likelihood of remaining in the same state, and indicating a lower acceptance rate. Conversely, lower β reduces the effect of the objective function on state probabilities, making states nearly equiprobable and indicating a higher acceptance rate.

Within the algorithm, a target acceptance rate is determined and β is adaptively adjusted to achieve the target acceptance rate. If the current acceptance rate exceeds the target, β is increased; if it falls below the target, β is decreased. This adjustment continues until the acceptance rate converges to the target rate. The initial β has minimal impact due to rapid updates during early iterations.

We successively reduce the initial target acceptance rate after a predetermined number of iterations. This allows for broader state exploration in the early stages of optimization, followed by a more greedy search for optimal results in later stages. This approach implements a special form of *simulated annealing* [33].

C. Algorithm Description

The pseudo-code for the general case of our MC^2 algorithm is presented in Algorithm 1. For simplicity, this algorithm assumes a constant target acceptance rate, but can be easily adapted for successive reduction in the target acceptance rate.

In the partitioning case, the algorithm operates on the list of short cycle candidates in the base matrix and tries to minimize the weighted sum of short cycle candidate counts. We typically assign weights $(w_4, w_6, w_8) = (0, 1, 0.2)$, prioritizing cycle-6 candidates while discarding cycle-4 candidates since cycles-4 are easily addressed at the lifting stage. The initial partitioning is derived from the GD algorithm [19], and the allowed state variables are constrained to be within L_1 and L_∞ distances from the initial state to limit the search space. Neighboring states violating this constraint are assigned with zero transition probability at each transition.

In the lifting case, the algorithm minimizes short cycle or common substructure counts. It can operate on the list of cycle candidates in protograph corresponding to $\mathbf{H}_{\text{SC}}^{\text{g}}$, starting with a random or basic arrangement that eliminates all cycles-4. It then minimizes the count of active cycle-6 candidates, and subsequently active cycle-8 candidates if all cycle-6 candidates are inactive, ensuring the removal of all shorter cycles. Alternatively, the algorithm can focus on reducing the number of unlabelled elementary absorbing or trapping sets (UAS/UTS) [12], such as the $(4, 4(\gamma - 2))$ UAS/UTS, while maintaining low-weight codewords and cycles-4 eliminated.[2]

[2]An (a, d_1) UTS is a configuration with a variable nodes (VNs), d_1 degree-1 check nodes (CNs), and no degree > 2 CNs. An (a, d_1) UAS is a UTS such that each VN is adjacent to strictly more degree-2 than degree-1 CNs. In a binary code, these become elementary trapping/absorbing sets [12].

Algorithm 1 Markov Chain Monte Carlo (MC^2) Optimizer for Object Count Reduction

Inputs: $\mathcal{L}$: list of objects of interest; d: cardinality of index tuples; $\mathbf{x}_{\text{init}}$: initial input vector; β_{init}: initial value of β; $\mathcal{T}$: maximum transition count (maximum number of iterations); $\mathbf{a}$: set of possible values for each entry of the input vector; f_T: target acceptance rate.

Outputs: $\overline{C}_{\text{opt}}$: minimum value of the objective function recorded during iterations; $\mathbf{x}_{\text{opt}}$: optimal value of the input vector resulting in the minimum objective function; i: number of transitions.

Intermediate Variables: S: list of index d-tuples; $\mathbf{x}$: input vector corresponding to the current state of the Markov chain; i_a: number of transitions to distinct states; β: hyper-parameter of the main distribution; ν: index tuple of entries that are updated; Z: normalizing factor for probabilities; $\mathbf{x}', \mathbf{x}_{\text{prev}}$: input vectors used for intermediate calculations; P_ν, P_ν^*: normalized and non-normalized mappings of conditional PMF values of the transition probability $P(\mathbf{x}'|\mathbf{x})$ for the current indices ν such that $\mathbf{x}'$ and $\mathbf{x}$ differ only at entries indexed by ν; f: acceptance rate.

1: Initialize S by going over $\mathcal{L}$ and calculating the common object counts for all entry pairs, then list d indices of the most correlated entries for each entry.
2: $\mathbf{x} \leftarrow \mathbf{x}_{\text{init}}$.
3: Initialize $\overline{C}_{\text{opt}} \leftarrow 1$, $(i, i_a) \leftarrow (0, 0)$, $\beta \leftarrow \beta_{\text{init}}$.
4: **while** $i < \mathcal{T}$ **do**
5: Shuffle the order of S randomly.
6: **for each** $\nu \in S$ **do**
7: $i \leftarrow i + 1$, $Z \leftarrow 0$, $\mathbf{x}_{\text{prev}} \leftarrow \mathbf{x}$.
8: **for each** $\mathbf{x}'_\nu \in \mathbf{a}^d$ **do**
9: Merge $\mathbf{x}'_\nu$ and $\mathbf{x}_{\setminus\nu}$ to obtain $\mathbf{x}'$. *// Obtain entries at indices ν from $\mathbf{x}'_\nu$ and the rest from $\mathbf{x}_{\setminus\nu}$.*
10: **if** $\mathbf{x}'$ does not satisfy the constraints **then**
11: $P_\nu(\mathbf{x}') \leftarrow 0$.
12: **else**
13: Evaluate $\overline{C}(\mathbf{x}')$.
14: **if** $\overline{C}(\mathbf{x}') < \overline{C}_{\text{opt}}$ **then**
15: $\overline{C}_{\text{opt}} \leftarrow \overline{C}(\mathbf{x}')$ and $\mathbf{x}_{\text{opt}} \leftarrow \mathbf{x}'$.
16: **if** $\overline{C}_{\text{opt}} = 0$ **then go to** Step 30.
17: **end if**
18: **end if**
19: $P_\nu^*(\mathbf{x}') \leftarrow \exp(-\beta \overline{C}(\mathbf{x}'))$.
20: $Z \leftarrow Z + P_\nu^*(\mathbf{x}')$.
21: **end if**
22: **end for**
23: $P_\nu \leftarrow P_\nu^*/Z$. *// This applies for all $\mathbf{x}$.*
24: $\mathbf{x} \leftarrow \text{overrelaxSampling}(P_\nu)$.
25: **if** $\mathbf{x} \neq \mathbf{x}_{\text{prev}}$ **then** $i_a \leftarrow i_a + 1$.
26: **end if**
27: **end for**
28: $f = i_a/i$, $\beta \leftarrow \text{updateBeta}(\beta, f, f_T)$. *// updateBeta and overrelaxSampling are functions we define.*
29: **end while**
30: **return** $\overline{C}_{\text{opt}}$, $\mathbf{x}_{\text{opt}}$, and i.

Two self-defined functions are used in the pseudo-code of the algorithm, described as follows:

1) $\mathrm{updateBeta}\,(\beta, f, f_T)$ adjusts β based on the current acceptance rate f and the target acceptance rate f_T. A proportional integral derivative (PID) controller is employed to increase β when $f > f_T$ and decrease it when $f < f_T$.
2) $\mathrm{overrelaxSampling}(P_\nu)$ generates samples from the PMF associated with P_ν using *ordered overrelaxation*, a method designed to mitigate the random walk behavior in Gibbs sampling [34], with little complexity overhead for each transition. It is estimated to accelerate the convergence of Gibbs sampling by a factor of 10 to 20.

The evaluation of the objective function involves iterating over the respective cycle candidate list and checking whether each cycle candidate is active or not. For partitioning, activeness is checked based on Equation (2), while for lifting, activeness is determined using Equation (3). As an additional case for the lifting stage, to reduce $(4, 4\,(\gamma - 2))$ UAS/UTS, the algorithm checks cycle-8 activeness and the existence of internal connections, counting only active cycle-8 candidates without any internal connections (see also [22]). For each case, these counts are normalized by the maximum value.

Remark 2: The MC^2 algorithm can also be adapted to target the more detrimental absorbing and trapping sets (ASs/TSs), which are key contributors to the error-floor phenomenon. These structures can be represented as disjunctive unions of fundamental cycles in a cycle basis [12], and they remain active only if all their fundamental cycles are active. Therefore, their patterns can be identified in the underlying graph along with their fundamental cycles, and their activeness can be assessed accordingly. The MC^2 algorithm can then be applied in a similar manner to minimize the number of ASs/TSs.

As mentioned earlier, evaluating conditional probabilities for a single transition requires $O(N^d)$ objective function evaluations, where N is the number of possible values per entry and d is the number of updated entries. Since each objective evaluation has a complexity of $O(|\mathcal{L}|)$, where $\mathcal{L}$ is the set of objects of interest, the overall complexity per transition is $O(N^d|\mathcal{L}|)$, and the total complexity is $O(\mathcal{T}N^d|\mathcal{L}|)$, with $\mathcal{T}$ denoting the maximum number of transitions.

To select d and the target acceptance rate, we perform a grid search over a small set of values given the code parameters during early iterations and proceed with the best-performing choice, which causes only a minor computational overhead. In our experiments, $\mathcal{T}$ is set between $2{,}000\gamma\kappa$ and $20{,}000\gamma\kappa$, which are affordable in our method, to ensure computational feasibility. Finally, the algorithm terminates when an input vector achieves an objective function value of zero.

IV. Data Fitting and Estimation Analysis

In this section, we discuss how data fitting can be applied to estimate the minimum value the Markov chain Monte Carlo (MC^2) algorithm can achieve and to provide a better understanding of its dynamics.

Definition 3 (Distribution of the Objective Function): We introduce the probability distribution of the objective function under the main distribution of recurrent class A as:

$$P_{\overline{C}_A}(\mathcal{C}) = \mathbb{P}\left[\overline{C}(\mathbf{x}) = \mathcal{C} \mid \mathbf{x} \in A\right] = \sum_{\mathbf{x}\in A;\, \overline{C}(\mathbf{x})=\mathcal{C}} P_A(\mathbf{x})$$
$$= \sum_{\mathbf{x}\in A;\, \overline{C}(\mathbf{x})=\mathcal{C}} \frac{e^{-\beta\overline{C}(\mathbf{x})}}{Z_A(\beta)} = \left|\{\mathbf{x} \mid \mathbf{x} \in A,\, \overline{C}(\mathbf{x}) = \mathcal{C}\}\right| \frac{e^{-\beta\mathcal{C}}}{Z_A(\beta)}. \tag{23}$$

Lemma 3: The mean and standard deviation of $\overline{C}_A(\mathbf{x})$, which represents $\overline{C}(\mathbf{x})$ given $\mathbf{x} \in A$, are denoted by $\mu_A(\beta)$ and $\sigma_A(\beta)$, functions of β, and they satisfy

$$\frac{d}{d\beta}\mu_A(\beta) = -\sigma_A^2(\beta).$$

Proof: The mean of $\overline{C}_A(\mathbf{x})$ can be written as

$$\mu_A(\beta) = \sum_{\mathbf{x}\in A} \overline{C}(\mathbf{x}) P_A(\mathbf{x}) = \sum_{\mathbf{x}\in A} \overline{C}(\mathbf{x}) \frac{e^{-\beta\,\overline{C}(\mathbf{x})}}{Z_A(\beta)}. \tag{24}$$

Using Equation (16), it can be shown that

$$\frac{d}{d\beta} Z_A(\beta) = -\sum_{\mathbf{x}\in A} \overline{C}(\mathbf{x})\, e^{-\beta\overline{C}(\mathbf{x})} = -Z_A(\beta)\, \mu_A(\beta). \tag{25}$$

Lastly, Equations (24) and (25) can be combined to show that

$$\frac{d}{d\beta}\mu_A(\beta) = \frac{d}{d\beta}\left(\sum_{\mathbf{x}\in A} \overline{C}(\mathbf{x}) \frac{e^{-\beta\,\overline{C}(\mathbf{x})}}{Z_A(\beta)}\right)$$
$$= \sum_{\mathbf{x}\in A} \overline{C}(\mathbf{x}) \frac{d}{d\beta}\left(\frac{e^{-\beta\,\overline{C}(\mathbf{x})}}{Z_A(\beta)}\right)$$
$$= \sum_{\mathbf{x}\in A} \left(-\overline{C}^2(\mathbf{x}) \frac{e^{-\beta\,\overline{C}(\mathbf{x})}}{Z_A(\beta)} + \overline{C}(\mathbf{x})\, \mu_A(\beta) \frac{e^{-\beta\,\overline{C}(\mathbf{x})}}{Z_A(\beta)}\right)$$
$$= -\left(\sum_{\mathbf{x}\in A} \overline{C}^2(\mathbf{x}) P_A(\mathbf{x})\right) + \mu_A(\beta)\left(\sum_{\mathbf{x}\in A} \overline{C}(\mathbf{x}) P_A(\mathbf{x})\right)$$
$$= -\left[\mathbb{E}_A\left[\overline{C}_A^2(\mathbf{x})\right] - \left(\mathbb{E}_A\left[\overline{C}_A(\mathbf{x})\right]\right)^2\right] = -\sigma_A^2(\beta). \tag{26}$$

■

To analyze the effect of β, we adapt the MC^2 optimization algorithm to generate a sequence of $\overline{C}(\mathbf{x})$ values for statistical analysis. The algorithm is executed for predetermined and fixed values of β. For each fixed β, we collect sequences, generate histograms of the collected data, and record the acceptance rate. We then fit the data to known probability distributions based on the generated histograms in order to evaluate the effect of β.

Our results show that for high object populations, the distribution of the objective function closely follows a discrete Gaussian distribution, reasonably approximated by a continuous Gaussian. We further observe that the acceptance rate $f(\beta)$ can be approximated by a decreasing degree-2 polynomial for $\beta \geq 0$ in the region of interest. Above a threshold for β, the Markov chain gets trapped in a local optimum, disrupting the fitting distribution. In this regime, $f(\beta)$ oscillates near zero, approximated in our remaining derivations as $f(\beta) \approx 0$.

It should be noted that the approximated continuous Gaussian distribution of the objective function should be clipped at $\overline{C}_A^*$, since the objective function cannot take smaller values.

For sufficiently small values of β, the probability of the objective function taking values below its minimum in an unclipped continuous Gaussian is infinitesimally small and can be neglected. Therefore, we use the unclipped Gaussian distribution to fit the collected data and select β values from a region where they are small enough to make the difference between the clipped and unclipped distributions negligible.

Remark 3: It has been shown in [20] and [35] that for short cycles, the probability of remaining active after random partitioning (resp., lifting) is of order $1/m$ (resp., $1/z$), under the assumption that the partitioning (resp., lifting) matrix entries are drawn from a uniform, independent, and identical distribution (i.i.d.). However, in the MC^2 framework, this assumption does not strictly hold, as the probability distribution depends on the number of active objects, which share non-zero entries, and therefore introduce dependencies.

At $\beta = 0$, the entry distribution is approximately uniform, since all feasible arrangements are equally likely, although the problem constraints still impose dependencies on the distribution. Let S_i denote a Bernoulli random variable representing the activeness of the i-th object in the object list, and let $C(\mathbf{x}) = \sum_i S_i$ denote the unnormalized objective function. For small β, it is reasonable to approximate the mean of S_i as $O(1/m)$ or $O(1/z)$, while acknowledging that the variables S_i are dependent due to shared entries among cycles. By the central limit theorem, the distribution of $C(\mathbf{x})$ can then be approximated as a Gaussian distribution, which aligns well with our empirical observations, particularly for large object sets. See Fig. 3 for the resemblance between the distribution of the numerical data generated by the MC^2 framework and the Gaussian distribution.

We consider the relationship between the mean and the variance of the fitted distributions. We realize that collected mean and variance pairs, as they vary with β, could be fitted to a linear relationship. This observation leads us to the equation:

$$\sigma_A^2(\beta) \approx k\mu_A(\beta) + l, \tag{27}$$

$\exists k > 0$ and $\exists l < 0$.

Combining Equations (26) and (27), we deduce the following differential equation for the mean:

$$\frac{d}{d\beta}\mu_A(\beta) = -\sigma_A^2(\beta) \approx -k\mu_A(\beta) - l. \tag{28}$$

The solution to this differential equation is given by:

$$\mu_A(\beta) \approx \left(\mu_0 + \frac{l}{k}\right) e^{-k\beta} - \frac{l}{k}, \tag{29}$$

where $\mu_0 = \mu_A(0)$.

Remark 4: An independent fit of the mean as a function of β, named as $\overline{\mu}_A(\beta)$, also closely conforms to the exponential decay model:

$$\overline{\mu}_A(\beta) = c + ae^{-b\beta}, \tag{30}$$

for $a, b, c > 0$ obtained by fitting, further validating the model as an accurate representation of the data. Based on this, we will subsequently use $\overline{\mu}_A(\beta)$ as an estimate of the mean of $\overline{C}_A(\mathbf{x})$ and naturally assume that it satisfies the properties of the actual mean.

Furthermore, we derive the expression for the estimated standard deviation of $\overline{C}_A(\mathbf{x})$ using Equations (26) and (30):

$$\overline{\sigma}_A(\beta) = \sqrt{-\frac{d}{d\beta}\overline{\mu}_A(\beta)} = \sqrt{ab}\, e^{-\frac{b}{2}\beta}. \tag{31}$$

Remark 5: It can be observed that increasing β gives more weight to lower $\overline{C}_A(\mathbf{x})$ values and their corresponding $\mathbf{x}$ vectors, for both the distribution of the objective function and the main distribution of the recurrent class A. Conversely, reducing β diminishes the effect of the objective function, making all $\mathbf{x}$ vectors close to equally likely. If we consider the two corner cases:

1) *When $\beta = 0$, all $\mathbf{x}$ vectors become equally likely, and the main distribution of the recurrent class A becomes uniform over all $\mathbf{x} \in A$.*
2) *For the other extreme, as $\beta \to \infty$, the probabilities of the optimal values dominate, making $P_A(\mathbf{x})$ uniform over A^* and zero elsewhere. Similarly, the distribution of the objective function converges to $\delta\left[\mathcal{C} - \overline{C}_A^*\right]$.*

Therefore, it can be expected that the mean of $\overline{C}_A(\mathbf{x})$ decreases as β increases, whereas the standard deviation is maximized when $\beta = 0$ and approaches zero as β goes to infinity. These observations align with the form of our estimated functions for these parameters, supportingour fitting results. Also, as $\beta \to \infty$, $P_{\overline{C}_A}(\mathcal{C})$ converges to $\delta\left[\mathcal{C} - \overline{C}_A^*\right]$ and

$$\overline{C}_A^* = \lim_{\beta\to\infty} \mu_A(\beta) \approx \lim_{\beta\to\infty} \overline{\mu}_A(\beta) = c, \tag{32}$$

showing that parameter c in the fitting function could be used as an estimate for the minimum of the objective function. We share the results of this estimate in Table III, Section V.

Additionally, we can use the outcomes of the fitting results to offer insights about the expected iteration count of the algorithm to reach a certain sub-minimum value within a given distance from the optimum, as well as the cardinality of the recurrent classes that the algorithm enters. First, we assume that the objective function distribution on the recurrent class A for different β values can be approximated as a continuous Gaussian distribution with a probability density function (PDF) given by

$$f_{\overline{C}_A}(\mathcal{C}) = \frac{1}{\sqrt{2\pi\overline{\sigma}_A^2(\beta)}} \exp\left(-\frac{(\mathcal{C} - \overline{\mu}_A(\beta))^2}{2\overline{\sigma}_A^2(\beta)}\right). \tag{33}$$

We assume that the fitting parameter corresponding to the estimation of the minimum value, c, is close enough to the actual minimum such that we can write

$$\overline{\mu}_A(\beta) = \overline{C}_A^* + a\, e^{-b\beta}. \tag{34}$$

Then, we can combine Equations (31), (33), and (34) to approximate the probability that the objective function is within a certain bounded distance, ϵ, from the optimum as

$$\mathbb{P}\left[\overline{C}_A(\mathbf{x}) - \overline{C}_A^* \le \epsilon\right] \approx \int_{\overline{C}_A^*}^{\overline{C}_A^*+\epsilon} f_{\overline{C}_A}(\mathcal{C})d\mathcal{C}$$
$$= Q\left(\frac{\overline{C}_A^* - \overline{\mu}_A(\beta)}{\overline{\sigma}_A(\beta)}\right) - Q\left(\frac{\overline{C}_A^* + \epsilon - \overline{\mu}_A(\beta)}{\overline{\sigma}_A(\beta)}\right)$$

$$= Q\left(-\sqrt{\frac{a}{b}}\, e^{-\frac{b}{2}\beta}\right) - Q\left(-\sqrt{\frac{a}{b}}\, e^{-\frac{b}{2}\beta} + \frac{\epsilon}{\sqrt{ab}}\, e^{\frac{b}{2}\beta}\right), \tag{35}$$

which we can evaluate after fitting, for any value of β and ϵ. Here $Q(x)$ is defined as:

$$Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty \exp\left(-\frac{u^2}{2}\right) du. \tag{36}$$

Furthermore, Equation (35) can be effectively approximated for most of the ranges of β and ϵ by replacing the Q function with the expression in Equation (37) below [36]

$$Q(x) = \frac{1}{\sqrt{2\pi}} \left[\frac{1}{\left(1-\frac{1}{\pi}\right)x + \frac{1}{\pi}\sqrt{x^2+2\pi}}\right] e^{-\frac{x^2}{2}}. \tag{37}$$

However, both Equations (35) and (37) are susceptible to floating-point precision errors for small values of β and ϵ. These operational ranges of β and ϵ become instrumental for certain estimation processes, which will be discussed later in this section. In such cases, a first order simplification using the Taylor series expansion of the second Q function in Equation (35) has the form

$$\mathbb{P}\left[\overline{C}_A(\mathbf{x}) - \overline{C}_A^* \leq \epsilon\right] \approx \frac{\epsilon}{\sqrt{2\pi ab}} \exp\left(\frac{b}{2}\beta - \frac{a}{2b}\, e^{-b\beta}\right), \tag{38}$$

and it is more resilient to floating-point errors and could serve as an effective alternative for small β and ϵ ranges.

This approximated probability can be used to estimate the order of the iteration count of the MC^2 algorithm with fixed β to reach a suboptimal value within difference ϵ from the optimum of A. This count is denoted by $\mathrm{iter}\,(\beta, \epsilon, A)$. We also introduce the set $A_\epsilon = \left\{\mathbf{x} \mid \overline{C}(\mathbf{x}) - \overline{C}_A^* \leq \epsilon, \mathbf{x} \in A\right\}$, which is the set that we are trying to sample from. For ideal sampling, i.e., if we assume a geometric distribution argument for the iteration count, $\mathbb{E}\left[\mathrm{iter}\,(\beta, \epsilon, A)\right]$ should be the inverse of $\mathbb{P}\left[\overline{C}_A(\mathbf{x}) - \overline{C}_A^* \leq \epsilon\right]$. However, our Markov chain does not behave as fluidly as an ideal sampler in terms of its transitions between consecutive states. When the transition rate is low, this can negatively impact the required number of iterations. To account for this, we incorporate an estimate of the acceptance rate, $\overline{f}(\beta)$, into the iteration count order estimate and inversely proportional to it such that this factor is reflected. Thus, the iteration count estimate is

$$\mathbb{E}\left[\mathrm{iter}\,(\beta, \epsilon, A)\right] \sim O\left(\frac{1}{\overline{f}(\beta)\,\mathbb{P}\left[\overline{C}_A(\mathbf{x}) - \overline{C}_A^* \leq \epsilon\right]}\right). \tag{39}$$

Observe that the iteration expectation is not directly equal to the number in the order argument, but it is expected that the iteration count is in this order.

Lastly, we can use Equation (35) to estimate the order of the cardinality of the recurrent class that the algorithm enters. We focus on the $\beta = 0$ case. It follows from Equation (16) that $Z_A(0) = |A|$. Additionally, $P_A(\mathbf{x})$ becomes uniform over A in this case. Therefore,

$$\mathbb{P}\left[\overline{C}_A(\mathbf{x}) - \overline{C}_A^* \leq \epsilon\right] = \frac{|A_\epsilon|}{|A|}. \tag{40}$$

Additionally, the expression in Equation (35) becomes

$$\mathbb{P}\left[\overline{C}_A(\mathbf{x}) - \overline{C}_A^* \leq \epsilon\right] \approx Q\left(-\sqrt{\frac{a}{b}}\right) - Q\left(-\sqrt{\frac{a}{b}} + \frac{\epsilon}{\sqrt{ab}}\right) \tag{41}$$

at $\beta = 0$. These two equations can be combined to reach

$$|A| = \frac{|A_\epsilon|}{Q\left(-\sqrt{\frac{a}{b}}\right) - Q\left(-\sqrt{\frac{a}{b}} + \frac{\epsilon}{\sqrt{ab}}\right)}. \tag{42}$$

For a small ϵ, we assume that $|A_\epsilon| \sim O(1)$, hence:

$$|A| \sim O\left(\frac{1}{Q\left(-\sqrt{\frac{a}{b}}\right) - Q\left(-\sqrt{\frac{a}{b}} + \frac{\epsilon}{\sqrt{ab}}\right)}\right). \tag{43}$$

Given that $C(\mathbf{x})$ takes integer values in the lifting stage, $\overline{C}(\mathbf{x})$ takes integer multiples of $1/\alpha$. Consequently, $1/\alpha$ is a suitable bin size for discretizing $f_{\overline{C}_A}(\mathcal{C})$ to approximate $P_{\overline{C}_A}(\mathcal{C})$. Because of that, we set $\epsilon = 1/\alpha$ for the estimations at the lifting stage. Combining this with Equations (38), and (43), we obtain:

$$|A| \sim O\left(\alpha\sqrt{2\pi ab}\, \exp\left(\frac{a}{2b}\right)\right). \tag{44}$$

We can conduct a similar analysis as well for the partitioning case, which we skip for brevity.

Observe that variations in β alter the value of transition probabilities in the Markov chain but do not affect the connectivity of states, and hence the recurrent classes, provided that β does not go to infinity. Therefore, Equation (44) is a valid estimate of the cardinality order for all finite β values.

Finally, we observed that reducing the sample size to a certain extent does not compromise parameter estimation or model fitting. We utilize 5 distinct β values, generating $100\gamma\kappa$ samples per β and resulting in a total of $500\gamma\kappa$ samples for estimation. The results of these estimations are presented in more detail in Section V.

V. Experimental Results

In this section, we present the performance evaluation of our Markov chain Monte Carlo (MC^2) optimizer, demonstrating the gains it offers across various codes compared with the literature in terms of reduced object counts and error/erasure-rate performance, and confirming the accuracy of the derived estimate based on data fitting. All codes here are binary codes.

We used optimal overlap (OO) partitioning [10] for low memory codes ($m \leq 2$) and for topologically-coupled (TC) codes [19]. Otherwise, gradient-descent supported MC^2 (GD-MC^2) algorithm is used for partitioning. For all of the codes, circulant power optimization is done by the MC^2 algorithm.

In Table I, we list parameters, lengths, and design rates for six spatially-coupled (SC) codes and one TC code. Our code list covers a wide range of lengths and design rates, demonstrating that the proposed method is effective for designing codes suitable for diverse applications, including communication systems. Construction methods and counts of objects of interest for these codes are provided in Table II. All codes are free of cycles-4, and those listed under cycle-8 counts are also

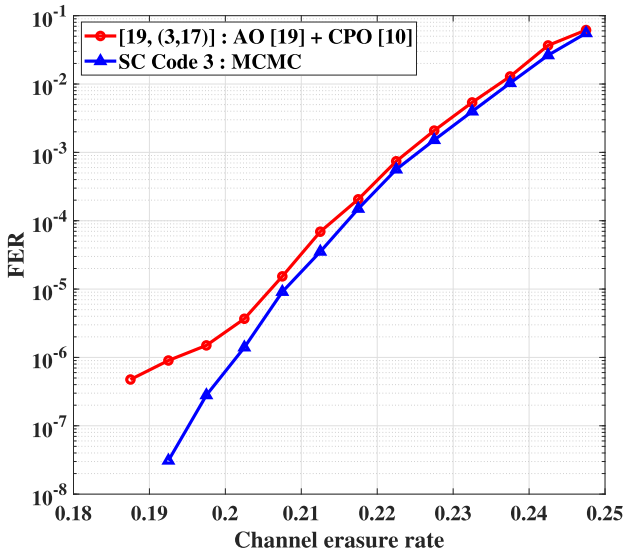

Fig. 1. FER curve of SC Code 3, along with its literature counterpart [19, $(3, 17)$], over the BEC. Both curves correspond to codes with parameters $(\gamma, \kappa, z, L, m) = (3, 17, 7, 10, 9)$.

free of cycles-6. We also refer the reader to literature papers from which we obtain some of our codes' partitioning matrix constructed by the OO method. We compare our codes with literature counterparts that share the same parameters, which are listed in Table I for all codes (thick lines in relevant tables separate pairs of codes we compare). We have modified the L parameter of the literature code [19, $(3, 17)$], while keeping the other code parameters and the partitioning/lifting matrices unchanged, in order to match its parameters with those of SC Code 3. The rationale for using a lower L is to obtain shorter codes suitable for wireless communication applications, thereby demonstrating the versatility of our proposed MC^2 method. Note that the optimization process during the partitioning stage is not affected by the number of replicas, since our goal is to minimize the number of active cycle candidates, which is independent of L. We also conducted the lifting-stage optimization separately for high and low L values, and observed that the resulting cycle counts differ by less than 1%, indicating that the choice of L only has a negligible impact on this stage.

Remark 6: Our MC^2 method can also be adapted for finite-length (FL) optimization of multi-dimensional (MD) codes via managing relocations to reduce short cycle counts. It can be integrated with probabilistic frameworks [20] similarly to the case of partitioning. To demonstrate this, we included MD-SC Code 1 with parameters $(\gamma, \kappa, z, L, m) = (4, 17, 17, 10, 1)$ *and* 3 *auxiliary matrices, resulting in a code of length* 8,670 *and rate* 0.74*. Partitioning is done using the OO method; lifting and relocations are handled by the MC^2 method, targeting cycles-*6*. MD-SC Code 1 has* 3,366 *cycles-*6*, compared with* 6,375 *and* 9,078 *in two literature codes [20], [31] with the same parameters and partitioning matrix, showing the effectiveness of our approach.*

We also present the actual and estimated minimum object counts, as well as the recurrent class cardinality order estimations (on base-$z \log$ scale) derived from data fitting at the lifting stage in Table III. We exclude SC Code 2 and SC Code 5 from such estimations since the MC^2 algorithm is able to remove all the objects of interest.

We compare the computational complexity of our method with literature methods for both partitioning and lifting. Table IV presents the number of evaluations required for each input to compute objective function values and check constraint satisfaction. We focus on the evaluations needed to achieve results close to those we report. For our method, the evaluation counts are represented in split cells: the left value indicates evaluations required to reach our optimal result, while the right value shows evaluations required to match literature results. For SC Code 2 and SC Code 5, we only share one count since we only consider the evaluation count to remove all objects of interest. Lastly, evaluation counts of partitioning are only reported for codes constructed using GD-based methods, since the OO method gives the final FL partitioning, and thus, no algorithmic optimization is required for this stage.

We also compare the computational latency of two of our code constructions with existing literature, focusing specifically on the run-times of algorithms. Relevant results are presented in Table V, where we list termination times measured in minutes. For each case, we provide both the time taken to reach our optimal solution in the left cell, and time to match the result from the literature in the right cell. For reproducibility, all software implementations were executed in MATLAB on a Lenovo ThinkSystem SR650. This system is

TABLE I

PARAMETERS, LENGTHS, AND RATES OF PROPOSED SC/TC CODES

Code name	γ	κ	z	L	m	Length	Rate
SC Code 1	3	17	17	30	1	8,670	0.82
SC Code 2	3	17	17	30	2	8,670	0.81
SC Code 3	3	17	7	10	9	1,190	0.66
SC Code 4	4	29	29	20	19	16,820	0.73
SC Code 5	3	7	11	30	5	2,310	0.50
SC Code 6	4	17	37	10	1	6,290	0.74
TC Code 1	4	17	17	50	4	14,450	0.75

TABLE II

PARTITIONING/LIFTING/RELOCATION METHODS, AND REDUCED OBJECT COUNTS OF PROPOSED SC/TC CODES (CODE PARAMETERS ARE LISTED IN TABLE I)

Code name	Partitioning	Lifting	Cycle-6 count
[10, SC Code 3]	OO [10]	[10]	14,960
SC Code 1	OO [10]	MC^2	12,937
[10, SC Code 7]	OO [10]	[10]	0
SC Code 2	OO [10]	MC^2	0
[19, $(4, 17)$]	TC-OO [19]	[19]	15,436
TC Code 1	TC-OO [19]	MC^2	8,007

Code name	Partitioning	Lifting	Cycle-8 count
[19, $(3, 17)$]	GRADE-AO [19]	[19]	13,860
SC Code 3	GD-MC^2	MC^2	12,530
[19, $(4, 29)$]	GRADE-AO [19]	[19]	528,090
SC Code 4	GD-MC^2	MC^2	409,973
SC Code 5	GD-MC^2	MC^2	0

Code name	Partitioning	Lifting	$(4, 8)$ UTS count
[22, Code 10]	OO [22]	[22]	1,253,177
SC Code 6	OO [22]	MC^2	1,229,177

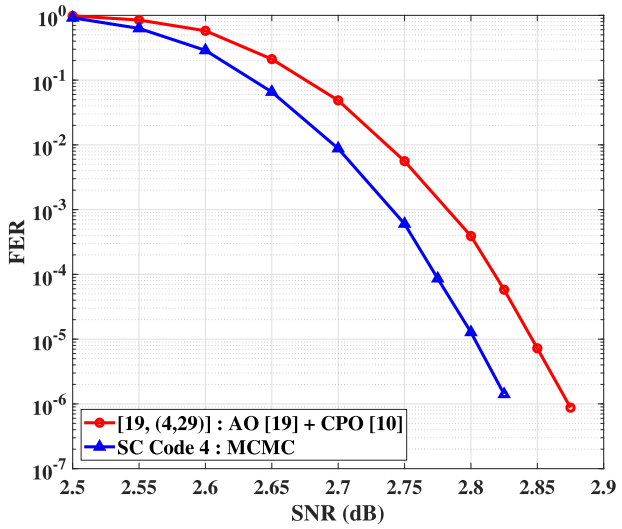

Fig. 2. FER curve of SC Code 4, along with its literature counterpart [19, $(4, 29)$], over the AWGNC. Both curves correspond to codes with parameters $(\gamma, \kappa, z, L, m) = (4, 29, 29, 20, 19)$.

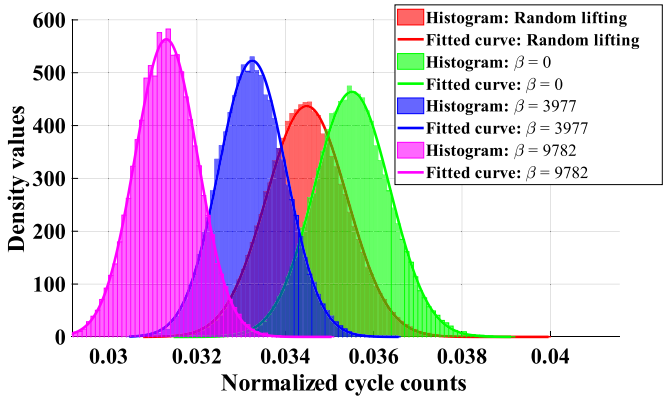

Fig. 3. Data acquisition and fitting results for SC Code 4 at the lifting stage. The code parameters are $(\gamma, \kappa, z, L, m) = (4, 29, 29, 20, 19)$. Three data sets were collected for different β values (0, 3977, and 9782), and they are shown in green, blue, and magenta, respectively. For comparison, a fourth curve is included, which is obtained by collecting cycle counts resulting from lifting matrices whose entries are drawn from a uniform i.i.d. (shown in red).

equipped with an Intel Xeon Gold 6226R CPU at 2.90 GHz with 64 cores.

SC Code 3 and SC Code 4, along with their literature counterparts, are simulated over the binary erasure channel (BEC) and the additive white Gaussian noise channel (AWGNC) using a peeling decoder and a sum-product (fast Fourier transform based) decoder [19], respectively. The results are presented in Fig. 1 and Fig. 2.

Lastly, we carried out the data acquisition and fitting process described in Section IV for SC Code 4 at the lifting stage. In particular, we collected cycle-8 count data for three different β values (0, 3977, and 9782), and fitted the resulting histograms of the collected data to Gaussian distributions. The fitted curves and the corresponding histograms are shown in Fig. 3. For comparison, we also included the case where a set of lifting matrices were generated randomly, with each matrix entry drawn from a uniform, independent, and identical

TABLE III

ESTIMATED AND ACTUAL MINIMUM RESULTS WITH RECURRENT CLASS CARDINALITY ESTIMATIONS (CODE PARAMETERS ARE LISTED IN TABLE I)

Code name	Minimum object count	Estimated object count	Recurrent class cardinality estimation
SC Code 1	12,937	13,399	14.70
SC Code 3	12,530	12,096	13.31
SC Code 4	409,973	390,953	29.51
SC Code 6	1,229,177	1,175,229	30.73
TC Code 1	8,007	11,757	17.63

TABLE IV

EVALUATION COUNTS AT PARTITIONING AND LIFTING STAGES (CODE PARAMETERS ARE LISTED IN TABLE I)

Code name	Partitioning		Lifting	
	Best result	Matched	Best result	Matched
[10, SC Code 3]	–		8,519,142	
SC Code 1	–		3,445,562	1,872,196
[10, SC Code 7]	–		1,586,051	
SC Code 2	–		561,162	
[19, $(3, 17)$]	3,060		12,630	
SC Code 3	218,490	5,304	2,663,153	386
[19, $(4, 29)$]	11,020		71,085,008	
SC Code 4	203,009	2,855	17,282,611	41,515
SC Code 5	33,369		7,613	
[22, Code 10]	–		54,350,669	
SC Code 6	–		520,090	85,816
[19, $(4, 17)$]	–		34,442,586	
TC Code 1	–		75,822,980	1,312,451

TABLE V

RUN-TIMES OF THE ALGORITHMS (CODE PARAMETERS ARE LISTED IN TABLE I)

Code name	Design stage	Run-time (in minutes)	
		Best result	Matched
[10, SC Code 3]	Lifting	70.03	
SC Code 1	Lifting	0.20	0.12
[19, $(4, 29)$]	Lifting	293.409	
SC Code 4	Lifting	10.20	0.15

distribution (i.i.d.), and the resulting normalized cycle-8 counts were collected. It can be observed from these plots that the distribution of the collected data closely resembles a Gaussian distribution, as mentioned earlier, in all the cases. The plots illustrate the change in the mean and variance of the distribution with increasing β, as discussed in Remark 5. The plots also highlight the differences between the distributions produced by the MC^2 framework and the uniform i.i.d. case, where each distribution exhibits distinct mean and variance characteristics.

Here are some discussions summarizing the main conclusions from our numerical results:

1) **Reduction of short cycle and common substructure counts:** The MC^2 algorithm effectively reduces the number of objects of interest, including cycles-6, cycles-8, and $(4, 8)$ UTSs, outperforming literature methods:
   - For cycles-6, count reductions range from 13.52% to 48.13% compared with respective literature counts. For instance, MD-SC Code 1 has a reduced count

of 3,366 compared with 6,375 (47.20%), demonstrating the gains of the algorithm when applied for lifting and relocations, while TC Code 1 achieves a 48.13% reduction.

- Cycle-8 count reductions range from 9.60% to 22.37%. Notably, SC Code 3 has 9.60% less objects than its counterpart, and SC Code 4 has 118,117 less objects. Our MC^2 algorithm is applied at both the partitioning and lifting stages for these codes, demonstrating the algorithm effectiveness and the remarkable gains it can achieve at both stages of the code design.
- For $(4, 8)$ UTS objects, SC Code 6 achieves a 1.92% reduction, corresponding to 24,000 fewer objects than the best literature counterpart. This MC^2 result is reached at a notably less run-time compared with the respective literature technique.
- For SC Code 5, a girth-10 code, the MC^2 algorithm eliminated all cycles of length ≤ 8, while the code rate is kept at a moderate 0.50.

These results highlight the versatility of the proposed MC^2 method in reducing various types of harmful structures, as well as its potential as a robust FL optimizer capable of minimizing more complex and detrimental objects beyond short cycles. They also demonstrate its applicability across different design stages and for various types of codes.

2) **Estimation accuracy:** Our fitting approach demonstrates high precision in estimating minimum object counts the MC^2 algorithm can reach, with deviations between estimated and actual results being consistently low. In particular, such deviations range from 3.59% to 6.23%, except for an isolated case with TC Code 1, where the deviation is significantly higher at 46.83%. For instance, the estimated cycle-6 count for SC Code 1 is 13,399, whereas the actual count is 12,937, resulting in a deviation of just 3.57%. This accuracy highlights the reliability of our predictions regarding the resulting code. It should also be noted that these estimation results are obtained with far fewer iterations than what the minimization algorithm uses, and yet, they still provide valuable insights into the algorithm behaviour.

3) **Computational complexity:** The MC^2 algorithm consistently achieves superior results (in the overwhelming majority of cases) or comparable results (in very few cases) in minimizing object counts. Typically, to reach the same literature count, our MC^2 algorithm offers orders of magnitude time-complexity reduction. Moreover, even to notably overcome the literature result, the MC^2 algorithm typically offers at most the same complexity, with various cases exhibiting remarkable complexity reduction as well. Key observations include:

   - For SC Code 1, SC Code 2, SC Code 4 (lifting stage only for this code), and SC Code 7, our algorithm shows better complexity outcomes, since it requires 0.4 to 2.0 orders of magnitude fewer evaluations to reach its minimum (which is a better result) compared with literature methods, and 0.45 to 3.23 orders of magnitude fewer evaluations to replicate the literature results.
   - For SC Code 3 (lifting stage only), SC Code 4 (partitioning stage only), and TC Code 1, our algorithm reaches the best results of literature techniques with 0.59 to 1.51 orders of magnitude fewer evaluations. Achieving its optimal results may require additional number of evaluations, making its complexity comparable to or slightly higher than literature methods for these specific codes, but with notably reduced object counts.
   - For SC Code 3 (partitioning stage only), our algorithm requires higher number of evaluations of objective function to achieve both its own best result and the literature best. However, the resulting code has significantly fewer objects with only a modest increase in complexity. It should be noted that this is the only case we encountered in which achieving the best result in literature requires a minor complexity increase by our method.

   Additionally, the MC^2 algorithm is highly suitable for parallelization, since the most computationally demanding task (i.e., evaluation of objective function) can be distributed across multiple threads. Efficiency of this approach, in terms of the required run-time, is evident in Table V. For both SC Code 1 and SC Code 4, our method achieves significantly better results in terms of run-time compared with the methods reported in the literature, both to match and even to outperform the literature counts.

4) **Improved simulation results:**

   - For SC Code 3, the literature code exhibits an error floor, whereas our code does not, thanks to the reduced number of detrimental objects, resulting in up to 1.46 orders of magnitude improvement in FER at a channel erasure rate of 0.1925. Furthermore, we collected and analyzed the frames with uncorrected erasures of the literature code in the error-floor region, and observed that a dominant size-10 stopping set is responsible for the majority of decoding failures.[3] This configuration is shown in Fig. 4, and it can be observed that it encompasses five cycles-8. Our code does not contain such a detrimental configuration, demonstrating that the MC^2 optimization we performed to reduce the number of cycles-8 significantly improved our code performance, resulting in no visible error floor.
   - SC Code 4 consistently outperforms its counterpart across the entire SNR range of interest, particularly in the waterfall region, due to the reduced number

[3]A size-a stopping set is a configuration with a variable nodes (VNs) and with check nodes (CNs) each of degrees at least 2. Stopping sets are the main cause of decoding failures for the peeling decoder [37].

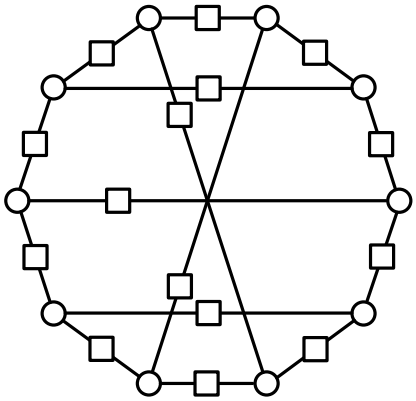

Fig. 4. A size-10 stopping set in the Tanner graph of the literature counterpart of SC Code 3, labeled as [19, (3, 17)], with parameters $(\gamma, \kappa, z, L, m) = (3, 17, 7, 10, 9)$. This configuration is identified as the main reason behind the faulty frames in the error-floor region. Note that here, circles represent VNs and squares represent CNs.

of detrimental objects, which leads to less message-passing dependencies. In this case, the maximum observed gain in FER reaches 1.61 orders of magnitude at a signal-to-noise ratio (SNR) of 2.825 dB. We also observe that at 10^{-6} FER level, our code has about 0.05 dB gain compared with its literature counterpart.

These results confirm that the FL optimizations achieved by the proposed MC^2 method leads to substantial practical performance improvements across different channels and decoding algorithms compared with available methods.

5) **Ergodicity and number of recurrent classes of the Markov chain:** Although it is not formally proven, our optimization problem is highly unlikely to be convex. Consequently, the Markov chain constructed for the algorithm is unlikely to possess a single recurrent class. Instead, it is expected to have multiple recurrent classes. From our fitting and estimation results, we observed that the cardinality of the recurrent classes where data is collected is in the order of $O(N^k)$, where k is smaller than $\gamma\kappa$ but larger than d, the number of entries updated at each transition. This empirical observation, shown in Table III, suggests that the Markov chains are non-ergodic and possess multiple recurrent classes each. Nevertheless, it has been shown that the observed recurrent class during the algorithm, considered as a small chain, satisfies the convergence condition of its main distribution. Furthermore, based on Theorem 1, it is expected that the optimal solution reached by the algorithm will be arbitrarily close to the optimal solution of this small chain under analysis.
6) **Suboptimal recurrent classes and compatibility with a GD-based probabilistic approach:** The performance of the MC^2 minimizer heavily depends on the recurrent class it enters, which influences the size of the search space, computational complexity, and the closeness of the reached minimum to the global minimum. Proper input vector initialization is therefore crucial.

TABLE VI

RESULTING CYCLE-8 COUNTS OF CODES INITIALIZED WITH UNIFORM OR GRADIENT-DESCENT-BASED DISTRIBUTION AT PARTITIONING STAGE (CODE PARAMETERS ARE LISTED IN TABLE I)

Code name	GD partitioning	UNF partitioning
[19, (3, 17)]	13,860	24,304
SC Code 3	12,530	28,567
[19, (4, 29)]	528,090	1,087,268
SC Code 4	409,973	1,191,059

GD-based initialization proves highly effective, directing the optimizer toward recurrent classes with better local minima. As shown in Table VI, GD-initialized runs achieve significantly lower cycle counts compared with those initialized with a uniform distribution (UNF). As intriguing, the performance gap is wider in favor of our algorithm in such situations compared with corresponding GD-based method in the literature [19]. In the case of lifting, a practical strategy is to perform multiple random initializations, execute the algorithm for a limited number of iterations in each run, and then proceed with the best-performing result to reduce the risk of being trapped in a poor recurrent class (poor local optimum).

7) **Generality of the MC^2 method as a finite-length optimizer:** Although this paper focuses on SC/TC code design during the partitioning and lifting stages, the MC^2-based approach can be readily adapted to other finite-length optimization problems in LDPC code design or, more broadly, in coding theory.
This method can be generalized for lifting arrangements of any type of LDPC codes beyond SC codes, applied to MD code design at the relocation stage, or even extended to certain cases of non-binary code design involving edge-weight arrangements. In addition to its minimization algorithm, the estimation and data-fitting components of the MC^2 approach can also be leveraged in various areas of code design, offering valuable insights into the theoretical aspects of the algorithm.

VI. CONCLUSION

Finite-length (FL) optimization in spatially-coupled (SC) code design is widely considered a computationally-taxing task, especially as more degrees of freedom become available. This work proposes a Markov chain Monte Carlo (MC^2)-based framework to address this discrete optimization problem, targeting both the partitioning and lifting stages to minimize the number of short cycles and common subgraphs of dominant absorbing sets. The method adopts a probabilistic approach to efficiently search for optimal or near-optimal solutions, while also enabling statistical analysis to estimate the resulting object counts, which is an interesting problem to tackle in its own right. Our numerical results demonstrate that the MC^2 approach consistently outperforms existing methods in terms of object count reduction, which reaches 48%. Our approach also offers superior computational efficiency, which reaches 3.32 orders of magnitude complexity reduction, compared with the literature in the vast majority of cases examined. Our

experimental findings demonstrate that the FL optimization achieved by the MC^2 method results in notable improvements in practical performance across a range of channels and decoders. In particular, simulation results reveal frame error/erasure-rate gains of up to 1.61 orders of magnitude compared with literature methods. Moreover, a general consistency is observed between the estimated and actual outcomes. Integrating this approach with complementary probabilistic frameworks, such as the gradient-descent algorithm in [19], results in additional performance gains. The proposed framework is expected to be applicable beyond SC code design, offering potential solutions for a broader range of discrete optimization problems in LDPC code design, and more generally, in coding theory. Future work includes applying the method to multi-dimensional SC code design as well as targeting more advanced and more detrimental structures, such as absorbing and trapping sets, to further enhance code performance.

REFERENCES

[1] R. G. Gallager, *Low-Density Parity-Check Codes*. Cambridge, MA, USA: MIT Press, 1963.

[2] D. J. C. MacKay, "Good error-correcting codes based on very sparse matrices," *IEEE Trans. Inf. Theory*, vol. 45, no. 2, pp. 399–431, Mar. 1999.

[3] A. Hareedy, H. Esfahanizadeh, and L. Dolecek, "High performance non-binary spatially-coupled codes for flash memories," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Kaohsiung, Taiwan, Nov. 2017, pp. 229–233.

[4] P. Chen, C. Kui, L. Kong, Z. Chen, and M. Zhang, "Non-binary protograph-based LDPC codes for 2-D-ISI magnetic recording channels," *IEEE Trans. Magn.*, vol. 53, no. 11, pp. 1–5, Nov. 2017.

[5] H. Esfahanizadeh, E. Ram, Y. Cassuto, and L. Dolecek, "A unified, SNR-aware SC-LDPC code design with applications to magnetic recording," *IEEE Trans. Magn.*, vol. 59, no. 3, pp. 1–9, Mar. 2023.

[6] A. Hareedy, C. Lanka, and L. Dolecek, "A general non-binary LDPC code optimization framework suitable for dense flash memory and magnetic storage," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 9, pp. 2402–2415, Sep. 2016.

[7] H. Cui, F. Ghaffari, K. Le, D. Declercq, J. Lin, and Z. Wang, "Design of high-performance and area-efficient decoder for 5G LDPC codes," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 2, pp. 879–891, Feb. 2021.

[8] M. P. C. Fossorier, "Quasi-cyclic low-density parity-check codes from circulant permutation matrices," *IEEE Trans. Inf. Theory*, vol. 50, no. 8, pp. 1788–1793, Aug. 2004.

[9] D. G. M. Mitchell, M. Lentmaier, and D. J. Costello, "Spatially coupled LDPC codes constructed from protographs," *IEEE Trans. Inf. Theory*, vol. 61, no. 9, pp. 4866–4889, Sep. 2015.

[10] H. Esfahanizadeh, A. Hareedy, and L. Dolecek, "Finite-length construction of high performance spatially-coupled codes via optimized partitioning and lifting," *IEEE Trans. Commun.*, vol. 67, no. 1, pp. 3–16, Jan. 2019.

[11] L. Schmalen and K. Mahdaviani, "Laterally connected spatially coupled code chains for transmission over unstable parallel channels," in *Proc. Int. Symp. Turbo Codes Iterative Inf. Process. (ISTC)*, Bremen, Germany, Aug. 2014, pp. 77–81.

[12] A. Hareedy, R. Kuditipudi, and R. Calderbank, "Minimizing the number of detrimental objects in multi-dimensional graph-based codes," *IEEE Trans. Commun.*, vol. 68, no. 9, pp. 5299–5312, Sep. 2020.

[13] M. Battaglioni, A. Tasdighi, G. Cancellieri, F. Chiaraluce, and M. Baldi, "Design and analysis of time-invariant SC-LDPC convolutional codes with small constraint length," *IEEE Trans. Commun.*, vol. 66, no. 3, pp. 918–931, Mar. 2018.

[14] S. Naseri and A. H. Banihashemi, "Construction of time invariant spatially coupled LDPC codes free of small trapping sets," *IEEE Trans. Commun.*, vol. 69, no. 6, pp. 3485–3501, Jun. 2021.

[15] S. Kudekar, T. J. Richardson, and R. L. Urbanke, "Threshold saturation via spatial coupling: Why convolutional LDPC ensembles perform so well over the BEC," *IEEE Trans. Inf. Theory*, vol. 57, no. 2, pp. 803–834, Feb. 2011.

[16] S. Kudekar, T. Richardson, and R. L. Urbanke, "Spatially coupled ensembles universally achieve capacity under belief propagation," *IEEE Trans. Inf. Theory*, vol. 59, no. 12, pp. 7761–7813, Dec. 2013.

[17] A. R. Iyengar, M. Papaleo, P. H. Siegel, J. K. Wolf, A. Vanelli-Coralli, and G. E. Corazza, "Windowed decoding of protograph-based LDPC convolutional codes over erasure channels," *IEEE Trans. Inf. Theory*, vol. 58, no. 4, pp. 2303–2320, Apr. 2012.

[18] H.-Y. Kwak, J.-W. Kim, H. Park, and J.-S. No, "Optimization of SC-LDPC codes for window decoding with target window sizes," *IEEE Trans. Commun.*, vol. 70, no. 5, pp. 2924–2938, May 2022.

[19] S. Yang, A. Hareedy, R. Calderbank, and L. Dolecek, "Breaking the computational bottleneck: Probabilistic optimization of high-memory spatially-coupled codes," *IEEE Trans. Inf. Theory*, vol. 69, no. 2, pp. 886–909, Feb. 2023.

[20] C. İrimağzı, A. Tanrıkulu, and A. Hareedy, "Probabilistic design of multi-dimensional spatially-coupled codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Athens, Greece, Jul. 2024, pp. 653–658.

[21] L. Dolecek, Z. Zhang, V. Anantharam, M. J. Wainwright, and B. Nikolic, "Analysis of absorbing sets and fully absorbing sets of array-based LDPC codes," *IEEE Trans. Inf. Theory*, vol. 56, no. 1, pp. 181–201, Jan. 2010.

[22] A. Hareedy, R. Wu, and L. Dolecek, "A channel-aware combinatorial approach to design high performance spatially-coupled codes," *IEEE Trans. Inf. Theory*, vol. 66, no. 8, pp. 4834–4852, Aug. 2020.

[23] D. G. M. Mitchell and E. Rosnes, "Edge spreading design of high rate array-based SC-LDPC codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Aachen, Germany, Jun. 2017, pp. 2940–2944.

[24] M. Battaglioni, F. Chiaraluce, M. Baldi, and D. G. M. Mitchell, "Efficient search and elimination of harmful objects for the optimization of QC-SC-LDPC codes," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Waikoloa, HI, USA, Dec. 2019, pp. 1–6.

[25] M. Battaglioni, F. Chiaraluce, M. Baldi, M. Pacenti, and D. G. M. Mitchell, "Optimizing quasi-cyclic spatially coupled LDPC codes by eliminating harmful objects," *EURASIP J. Wireless Commun. Netw.*, vol. 2023, no. 1, pp. 1–29, Jul. 2023.

[26] D. J. C. MacKay, *Information Theory, Inference and Learning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2003.

[27] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *J. Chem. Phys.*, vol. 21, no. 6, pp. 1087–1092, Jun. 1953.

[28] W. K. Hastings, "Monte Carlo sampling methods using Markov chains and their applications," *Biometrika*, vol. 57, no. 1, pp. 97–109, Apr. 1970.

[29] T. Elperin, "Monte Carlo structural optimization in discrete variables with annealing algorithm," *Int. J. Numer. Methods Eng.*, vol. 26, no. 4, pp. 815–821, Apr. 1988.

[30] X. Li, X. Tang, C.-C. Wang, and X. Lin, "Gibbs-sampling-based optimization for the deployment of small cells in 3G heterogeneous networks," in *Proc. 11th Int. Symp. Workshops Model. Optim. Mobile, Ad Hoc Wireless Netw. (WiOpt)*, Tsukuba, Japan, May 2013, pp. 444–451.

[31] H. Esfahanizadeh, L. Tauz, and L. Dolecek, "Multi-dimensional spatially-coupled code design: Enhancing the cycle properties," *IEEE Trans. Commun.*, vol. 68, no. 5, pp. 2653–2666, May 2020.

[32] S. Geman and D. Geman, "Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images," *IEEE Trans. Pattern Anal. Mach. Intell.*, vols. PAMI-6, no. 6, pp. 721–741, Nov. 1984.

[33] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[34] R. M. Neal, "Suppressing random walks in Markov chain Monte Carlo using ordered overrelaxation," in *Learning in Graphical Models*. Cham, Switzerland: Springer, 1998, pp. 205–228.

[35] S. Naseri, A. Dehghan, and A. H. Banihashemi, "On average number of cycles in finite-length spatially coupled LDPC codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Espoo, Finland, Jun. 2022, pp. 378–383.

[36] A. Leon-Garcia, *Probability, Statistics, and Random Processes for Electrical Engineering*, 3rd ed., Upper Saddle River, NJ, USA: Pearson Education, 2017.

[37] A. Dehghan and A. H. Banihashemi, "Asymptotic average multiplicity of structures within different categories of trapping sets, absorbing sets, and stopping sets in random regular and irregular LDPC code ensembles," *IEEE Trans. Inf. Theory*, vol. 65, no. 10, pp. 6022–6043, Oct. 2019.

[38] Y. Hashemi and A. H. Banihashemi, "New characterization and efficient exhaustive search algorithm for leafless elementary trapping sets of variable-regular LDPC codes," *IEEE Trans. Inf. Theory*, vol. 62, no. 12, pp. 6713–6736, Dec. 2016.
[39] S. Naseri and A. H. Banihashemi, "Spatially coupled LDPC codes with small constraint length and low error floor," *IEEE Commun. Lett.*, vol. 24, no. 2, pp. 254–258, Feb. 2020.

Ata Tanrıkulu received the B.S. degree in Electrical and Electronics Engineering from Middle East Technical University (METU), Turkey, in 2024, and the B.S. degree in Computer Engineering from METU in 2025. He is currently pursuing the M.S. degree in the Department of Electrical and Electronics Engineering at METU with a focus on communications. His research interests include coding and information theory, particularly error-correction codes and their applications in reliable communications and data storage.

Mete Yıldırım is an undergraduate student in the Department of Electrical and Electronics Engineering at Middle East Technical University (METU), Turkey. His research interests include coding theory, particularly error-correction codes.

Ahmed Hareedy (Member, IEEE) is an Assistant Professor with the Department of Electrical and Electronics Engineering at Middle East Technical University (METU), Turkey. He is also an Affiliated Faculty Member with the Institute of Applied Mathematics at METU, Turkey. He is interested in research questions in coding/information theory that are fundamental to opportunities created by the current, unparalleled access to data and computing. He received the Bachelor and M.S. degrees in Electronics and Communications Engineering from Cairo University, Egypt, in 2006 and 2011, respectively. He received the Ph.D. degree in Electrical and Computer Engineering from the University of California, Los Angeles (UCLA), USA, in 2018. He was a Postdoctoral Associate with the Department of Electrical and Computer Engineering at Duke University, USA, between 2018 and 2021. He worked with Mentor Graphics Corporation (currently, Siemens EDA) between 2006 and 2014. He worked as an Error-Correction Coding Architect with Intel Corporation in the summers of 2015 and 2017.

Dr. Hareedy won the 2018–2019 Distinguished Ph.D. Dissertation Award in Signals and Systems from the Department of Electrical and Computer Engineering at UCLA. He is a recipient of the Best Paper Award from the 2015 IEEE Global Communications Conference (GLOBECOM), Selected Areas in Communications, Data Storage Track. He won the 2017–2018 Dissertation Year Fellowship (DYF) at UCLA. He won the 2016–2017 Electrical Engineering Henry Samueli Excellence in Teaching Award for teaching Probability and Statistics at UCLA. He is a recipient of the Memorable Paper Award from the 2018 Non-Volatile Memories Workshop (NVMW) in the area of devices, coding, and information theory. He is a recipient of the 2018–2019 Best Student Paper Award from the IEEE Data Storage Technical Committee (DSTC). He was awarded the TÜB ITAK 2232-B International Fellowship for Early Stage Researchers in 2022. He was recently a Guest Editor of the Special Issue on Data Storage of the IEEE BITS THE INFORMATION THEORY MAGAZINE.